

Handling Sparsity

Dr. Andy Barnes
apb41@bath.ac.uk
9.22 Wessex House

Where are we?

	Lecture A	Lecture B	Lab
Week 1	Introduction to NLP	Language Preprocessing	Genre Classification
Week 2	Document Representations	Handling Sparsity	Sentiment Analysis
Week 3	Static Word Embeddings	Contextual Word Embeddings	Machine Translation
Week 4	Part-of-speech Tagging	Syntax Parsing	Knowledge Graphs
Week 5	Classical Sequence Models	Neural Sequence Models	Text Generation

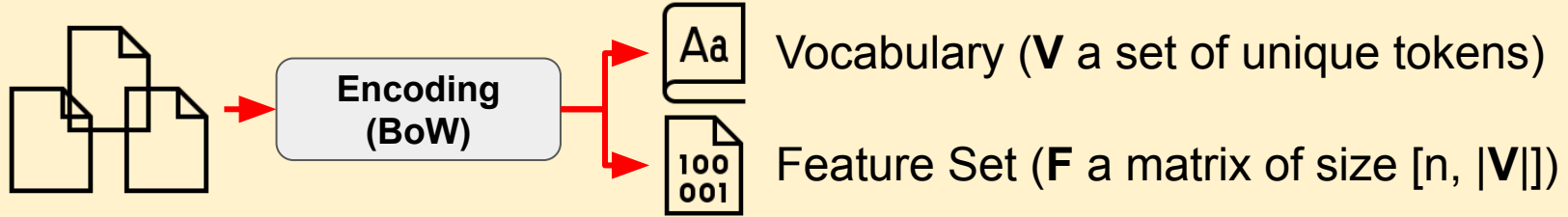
Learning Objectives

Describe when and how sparsity can hinder representations.

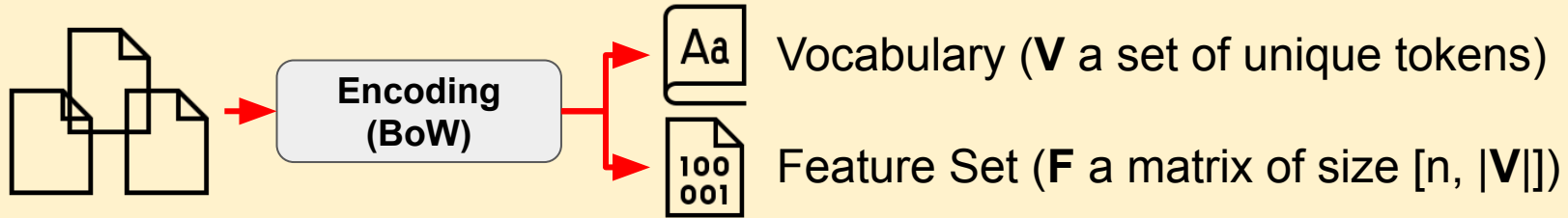
Explain at least three vocabulary reduction methods.

Apply two smoothing to improve probabilistic models.

Sparsity: What is it?



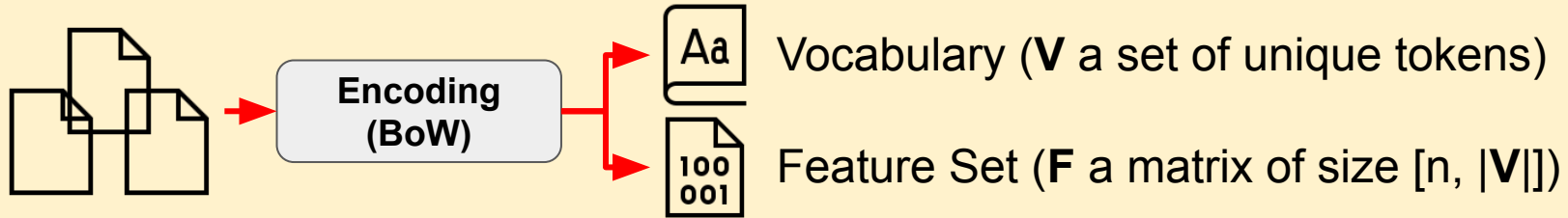
Sparsity: What is it?



Question

How does the size of the feature set scale with respect to the vocabulary?

Sparsity: What is it?

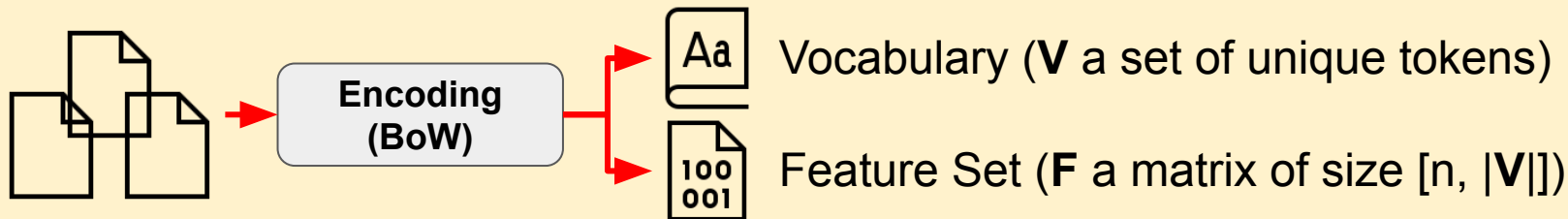


Question

How does the size of the feature set scale with respect to the vocabulary?

- Adding a token to **V** increases **F** by n elements.

Sparsity: What is it?



Question

How does the size of the feature set scale with respect to the vocabulary?

- Adding n tokens to **V** increases **F** by n elements.
- The problem with this is that a large vocabulary will often contain a lot of 0s.
 - This will cause most documents to be represented by a small fraction of the vocabulary.
 - For example, $|V|=100,000$, Avg. doc size = 75 unique words, 99% of feature entries will be 0.

Sparsity: Why do we care?

**Memory & Computational
Overhead**

Curse of dimensionality

Probabilistic Models

Sparsity: Why do we care?

Memory & Computational Overhead

Increasing the size of the vocabulary leads to the storage of more 0s, hence higher memory and computational requirements.

As we saw on the previous slide increasing $|\mathbf{V}|$ by a single token increases the size of the feature set by the number of documents (n).

When calculating distances, similarity etc. each feature is treated the same, so we need to be careful with resources.

Curse of dimensionality

Probabilistic Models

Sparsity: Why do we care?

Memory & Computational Overhead

Increasing the size of the vocabulary leads to the storage of more 0s, hence higher memory and computational requirements.

As we saw on the previous slide increasing $|V|$ by a single token increases the size of the feature set by the number of documents (n).

When calculating distances, similarity etc. each feature is treated the same, so we need to be careful with resources.

Curse of dimensionality

With increasing dimensions in a feature set distance metrics will lose their meaning.

To illustrate this, consider the trying to find your wallet in the following scenarios:

- Along a single street.
- In a park with interconnecting paths.
- In a carpark with multiple levels.

You remember 5 points in each of these you could have dropped it. No matter how you spread these out the search space gets sparser.

Probabilistic Models

Sparsity: Why do we care?

Memory & Computational Overhead

Increasing the size of the vocabulary leads to the storage of more 0s, hence higher memory and computational requirements.

As we saw on the previous slide increasing $|\mathbf{V}|$ by a single token increases the size of the feature set by the number of documents (n).

When calculating distances, similarity etc. each feature is treated the same, so we need to be careful with resources.

Curse of dimensionality

With increasing dimensions in a feature set distance metrics will lose their meaning.

To illustrate this, consider the trying to find your wallet in the following scenarios:

- Along a single street.
- In a park with interconnecting paths.
- In a carpark with multiple levels.

You remember 5 points in each of these you could have dropped it. No matter how you spread these out the search space gets sparser.

Probabilistic Models

The probabilities of most document-token pairings will collapse to 0.

Whilst this is true across all machine learning domains, it is especially prominent in NLP due to the size of some vocabularies.

This becomes more prominent in Naive-Bayes and other Bayesian methods for classification and generation.

Sparsity: Why do we care?

Memory & Computational Overhead

Increasing the size of the vocabulary leads to the storage of more 0s, hence higher memory and computational requirements.

As we saw on the previous slide increasing $|\mathbf{V}|$ by a single token increases the size of the feature set by the number of documents (n).

When calculating distances, similarity etc. each feature is treated the same, so we need to be careful with resources.

Curse of dimensionality

With increasing dimensions in a feature set distance metrics will lose their meaning.

To illustrate this, consider the trying to find your wallet in the following scenarios:

- Along a single street.
- In a park with interconnecting paths.
- In a carpark with multiple levels.

You remember 5 points in each of these you could have dropped it. No matter how you spread these out the search space gets sparser.

Probabilistic Models

The probabilities of most document-token pairings will collapse to 0.

Whilst this is true across all machine learning domains, it is especially prominent in NLP due to the size of some vocabularies.

This becomes more prominent in Naive-Bayes and other Bayesian methods for classification and generation.

Sparsity: Why do we care?

Question

Why would a sparse vocabulary hinder TF-IDF representations?

Memory & Computational Overhead

Increasing the size of the vocabulary leads to the storage of more 0s, hence higher memory and computational requirements.

As we saw on the previous slide increasing $|\mathbf{V}|$ by a single token increases the size of the feature set by the number of documents (n).

When calculating distances, similarity etc. each feature is treated the same, so we need to be careful with resources.

Curse of dimensionality

With increasing dimensions in a feature set distance metrics will lose their meaning.

To illustrate this, consider the trying to find your wallet in the following scenarios:

- Along a single street.
- In a park with interconnecting paths.
- In a carpark with multiple levels.

You remember 5 points in each of these you could have dropped it. No matter how you spread these out the search space gets sparser.

Probabilistic Models

The probabilities of most document-token pairings will collapse to 0.

Whilst this is true across all machine learning domains, it is especially prominent in NLP due to the size of some vocabularies.

This becomes more prominent in Naive-Bayes and other Bayesian methods for classification and generation.

Sparsity: Where does it come from?

Discrete Representations

One feature (dimension) per unique token, whether this is a unigram, trigram or simple one-hot encoding.

Sparsity: Where does it come from?

Discrete Representations

One feature (dimension) per unique token, whether this is a unigram, trigram or simple one-hot encoding.

Model Choices

The biggest contributor to large vocabularies will be the modelling choice.

For example, choosing trigram encoding scheme will create an exponentially larger vocabulary compared to unigram.

Sparsity: Where does it come from?

Discrete Representations

One feature (dimension) per unique token, whether this is a unigram, trigram or simple one-hot encoding.

Token Variability

Limited preprocessing will increase the size of the vocabulary by including stop-words, misspellings etc.

Model Choices

The biggest contributor to large vocabularies will be the modelling choice.

For example, choosing trigram encoding scheme will create an 'exponentially' larger vocabulary compared to unigram.

Sparsity: Where does it come from?

Discrete Representations

One feature (dimension) per unique token, whether this is a unigram, trigram or simple one-hot encoding.

Short Documents

If the document size is short in comparison to the vocabulary size each document will be a sparse representation.

(Unfortunately, there is no guide on this, some say document size should be at least the sqrt of $|V|$)

Token Variability

Limited preprocessing will increase the size of the vocabulary by including stop-words, misspellings etc.

Model Choices

The biggest contributor to large vocabularies will be the modelling choice.

For example, choosing trigram encoding scheme will create an exponentially larger vocabulary compared to unigram.

Sparsity: Where does it come from?

Natural Language

Human language tends to contain a small set of frequently used words and a large number of rare words.

Discrete Representations

One feature (dimension) per unique token, whether this is a unigram, trigram or simple one-hot encoding.

Short Documents

If the document size is short in comparison to the vocabulary size each document will be a sparse representation.

(Unfortunately, there is no guide on this, some say document size should be at least the sqrt of $|V|$)

Token Variability

Limited preprocessing will increase the size of the vocabulary by including stop-words, misspellings etc.

Model Choices

The biggest contributor to large vocabularies will be the modelling choice.

For example, choosing trigram encoding scheme will create an exponentially larger vocabulary compared to unigram.

COCA Frequencies:

- 'of' : 23,159,162
- 'do': 8,186,412
- ...
- 'oaken': 192
- 'dual': 48

<https://www.wordfrequency.info>

Sparsity: What can we do about it?

Vocabulary Pruning

Preprocessing

Stopword
Removal

Case-folding

Spelling
Correction

Stemming /
Lemmatisation

Sparsity: What can we do about it?

Vocabulary Pruning

Preprocessing

Stopword
Removal

Case-folding

Spelling
Correction

Stemming /
Lematisation

Frequency Pruning

Removing tokens which do not meet
a frequency of TF-IDF threshold.

Sparsity: What can we do about it?

Vocabulary Pruning

Preprocessing

Stopword
Removal

Case-folding

Spelling
Correction

Stemming /
Lemmatisation

Frequency Pruning

Removing tokens which do not meet
a frequency of TF-IDF threshold.

Tokenisation

Subword Encoding

BytePair

WordPiece

Byte-Level Tokens

Character Tokens

Sparsity: What can we do about it?

Vocabulary Pruning

Preprocessing

Stopword
Removal

Case-folding

Spelling
Correction

Stemming /
Lemmatisation

Frequency Pruning

Removing tokens which do not meet
a frequency or TF-IDF threshold.

Tokenisation

Subword Encoding

BytePair

WordPiece

Byte-Level Tokens

Character Tokens

Compound Splitting

Splitting out compound words.

notebook -> note + book

database = data + base

More useful in other languages (German,
Finnish etc.)

Sparsity: What can we do about it?

Vocabulary Pruning

Preprocessing

Stopword
Removal

Case-folding

Spelling
Correction

Stemming /
Lemmatisation

Frequency Pruning

Removing tokens which do not meet
a frequency of TF-IDF threshold.

Tokenisation

Subword Encoding

BytePair

WordPiece

Byte-Level Tokens

Character Tokens

Named Entity Normalisation

Replacing names, dates, times etc.
with placeholders.

'Luis was 10 minutes late to the 12:00
meeting'
'<PERSON> was <NUM> minutes late to the
<TIME> meeting'

Compound Splitting

Splitting out compound words.

notebook -> note + book

database = data + base

More useful in other languages (German,
Finnish etc.)

Sparsity: What can we do about it?

Question

Choose one of the pruning techniques and describe the potential downsides.

Vocabulary Pruning

Preprocessing

Stopword
Removal

Case-folding

Spelling
Correction

Stemming /
Lemmatisation

Frequency Pruning

Removing tokens which do not meet
a frequency of TF-IDF threshold.

Tokenisation

Subword Encoding

BytePair

WordPiece

Byte-Level Tokens

Character Tokens

Named Entity Normalisation

Replacing names, dates, times etc.
with placeholders.

'Luis was 10 minutes late to the 12:00
meeting'
'<PERSON> was <NUM> minutes late to the
<TIME> meeting'

Compound Splitting

Splitting out compound words.
notebook -> note + book
database = data + base
More useful in other languages (German,
Finnish etc.)

Sparsity: What can we do about it?

Vocabulary Pruning

Preprocessing

See lecture 2.

Frequency Pruning

Lose rare tokens
and document context

Tokenisation

Every tokenisation
scheme has
advantages and
disadvantages.

For example,
character level
tokenisation
requires models to
learn morphology.

Named Entity Normalisation

Requires additional compute to
determine NER, more on this in
Week 4.

Suffers from ambiguity (lexical and
contextual).

Compound Splitting

Non-compositional compound
words and idiomatic handling.

Data scarcity in some languages.

Sparsity: What can we do about it?

Probability Smoothing

- Building a Naive-Bayes spam classifier using unigrams.
 - Spam: ['buy', 'cheap', 'medicines'], ['cheap', 'bitcoin']
 - Not spam: ['client', 'meeting', 'morning'], ['cheap', 'meeting']

Sparsity: What can we do about it?

Probability Smoothing

- Building a Naive-Bayes spam classifier using unigrams.
 - Spam: ['buy', 'cheap', 'medicines'], ['cheap', 'bitcoin']
 - Not spam: ['client', 'meeting', 'morning'], ['cheap', 'meeting']

Token	Spam	Not Spam
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']



Sparsity: What can we do about it?

Probability Smoothing

- Building a Naive-Bayes spam classifier using unigrams.
 - Spam: ['buy', 'cheap', 'medicines'], ['cheap', 'bitcoin']
 - Not spam: ['client', 'meeting', 'morning'], ['cheap', 'meeting']

Token	Spam	Not Spam
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']



$$P(\text{Spam} | \text{'bitcoin meeting'}) \propto P(c) \cdot P(\text{'bitcoin'} | \text{Spam}) \cdot P(\text{'meeting'} | \text{Spam})$$

$$P(\text{NotSpam} | \text{'bitcoin meeting'}) \propto P(c) \cdot P(\text{'bitcoin'} | \text{NotSpam}) \cdot P(\text{'meeting'} | \text{NotSpam})$$

Sparsity: What can we do about it?

Question

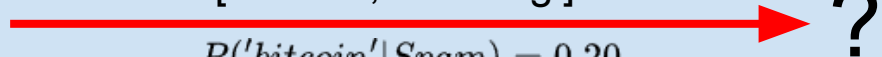
What happens when we try to resolve these equations?

Probability Smoothing

- Building a Naive-Bayes spam classifier using unigrams.
 - Spam: ['buy', 'cheap', 'medicines'], ['cheap', 'bitcoin']
 - Not spam: ['client', 'meeting', 'morning'], ['cheap', 'meeting']

Token	Spam	Not Spam
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']



$$P('bitcoin'|Spam) = 0.20$$

$$P('bitcoin'|NotSpam) = 0.00$$

$$P('meeting'|Spam) = 0.00$$

$$P('meeting'|NotSpam) = 0.33$$

$$P(Spam|'bitcoin meeting') \propto P(c) \cdot P('bitcoin'|Spam) \cdot P('meeting'|Spam)$$

$$P(NotSpam|'bitcoin meeting') \propto P(c) \cdot P('bitcoin'|NotSpam) \cdot P('meeting'|NotSpam)$$

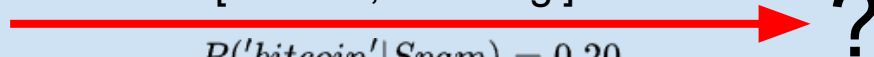
Sparsity: What can we do about it?

Probability Smoothing

- Building a Naive-Bayes spam classifier using unigrams.
 - Spam: ['buy', 'cheap', 'medicines'], ['cheap', 'bitcoin']
 - Not spam: ['client', 'meeting', 'morning'], ['cheap', 'meeting']

Token	Spam	Not Spam
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']



$$P('bitcoin'|Spam) = 0.20$$

$$P('bitcoin'|NotSpam) = 0.00$$

$$P('meeting'|Spam) = 0.00$$

$$P('meeting'|NotSpam) = 0.33$$

$$P(Spam|'bitcoin meeting') \propto 0.5 \cdot 0.20 \cdot 0.00$$

$$P(NotSpam|'bitcoin meeting') \propto 0.5 \cdot 0.00 \cdot 0.33$$

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']

Sparsity: What can we do about it?

Probability Smoothing

Laplace/Lidstone Smoothing

$$P(w|c) = \frac{\text{count}(w, c)}{T_c}$$

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']

Sparsity: What can we do about it?

Probability Smoothing

Laplace/Lidstone Smoothing

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{T_c + \alpha |\mathbf{V}|}$$

Alpha is a smoothing parameter:

- Lidstone: $0 < \alpha \leq 1$
 - Laplace: $\alpha = 1$

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']

Sparsity: What can we do about it?

Probability Smoothing

Laplace/Lidstone Smoothing

$$\cancel{P(w|c) = \frac{\text{count}(w, c)}{T_c}}$$

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{T_c + \alpha |\mathbf{V}|}$$

Alpha is a smoothing parameter:

- Lidstone: $0 < \alpha \leq 1$
 - Laplace: $\alpha = 1$

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']

$$P(\text{Spam} | \text{'bitcoin meeting'}) = 0.5 \cdot P(\text{'bitcoin'} | \text{Spam}) \cdot P(\text{'meeting'} | \text{Spam})$$

Sparsity: What can we do about it?

Probability Smoothing

Laplace/Lidstone Smoothing

$$\cancel{P(w|c) = \frac{\text{count}(w, c)}{T_c}}$$

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{T_c + \alpha |\mathbf{V}|}$$

Alpha is a smoothing parameter:

- Lidstone: $0 < \alpha \leq 1$
 - Laplace: $\alpha = 1$

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

['bitcoin', 'meeting']

$$P(\text{Spam} | \text{'bitcoin meeting'}) = 0.5 \cdot 0.160 \cdot 0.083$$

Sparsity: What can we do about it?

Probability Smoothing

Laplace/Lidstone Smoothing

$$\cancel{P(w|c) = \frac{\text{count}(w, c)}{T_c}}$$

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{T_c + \alpha |\mathbf{V}|}$$

Alpha is a smoothing parameter:

- Lidstone: $0 < \alpha \leq 1$
 - Laplace: $\alpha = 1$

$$P(\text{Spam} | \text{'bitcoin meeting'}) = 0.00664$$

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

[‘bitcoin’, ‘meeting’]

$$P(\text{NotSpam} | \text{'bitcoin meeting'}) = 0.5 \cdot P(\text{'bitcoin'} | \text{NotSpam}) \cdot P(\text{'meeting'} | \text{NotSpam})$$

Sparsity: What can we do about it?

Probability Smoothing

Laplace/Lidstone Smoothing

$$\cancel{P(w|c) = \frac{\text{count}(w, c)}{T_c}}$$

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{T_c + \alpha |\mathbf{V}|}$$

Alpha is a smoothing parameter:

- Lidstone: $0 < \alpha \leq 1$
 - Laplace: $\alpha = 1$

$$P(\text{Spam} | \text{'bitcoin meeting'}) = 0.00664$$

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

[‘bitcoin’, ‘meeting’]

$$P(\text{NotSpam} | \text{'bitcoin meeting'}) = 0.5 \cdot 0.083 \cdot 0.25$$

Sparsity: What can we do about it?

Probability Smoothing

Laplace/Lidstone Smoothing

$$\cancel{P(w|c) = \frac{\text{count}(w, c)}{T_c}}$$

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{T_c + \alpha |\mathbf{V}|}$$

Alpha is a smoothing parameter:

- Lidstone: $0 < \alpha \leq 1$
 - Laplace: $\alpha = 1$

$$P(\text{Spam} | \text{'bitcoin meeting'}) = 0.00664$$

$$P(\text{NotSpam} | \text{'bitcoin meeting'}) = 0.0104$$

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1

[‘bitcoin’, ‘meeting’]



Sparsity: What can we do about it?

Question

Use Laplace smoothing to classify:
[hello, prince, client, office]
with $\alpha = 0.5$.

Probability Smoothing

Laplace Smoothing

$$\cancel{P(w|c) = \frac{\text{count}(w, c)}{T_c}}$$

$$P(w|c) = \frac{\text{count}(w, c) + \alpha}{T_c + \alpha |\mathbf{V}|}$$

Alpha is a smoothing parameter:

- Lidstone: $0 < \alpha \leq 1$
 - Laplace: $\alpha = 1$

Token	S	NS
hello	15	15
prince	12	0
princess	8	0
client	12	14
french	0	10
london	3	3
office	0	8

Sparsity: What can we do about it?

Probability Smoothing

Witten-Bell Smoothing (Unigram)

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1
No. Tokens	5	5
Uniq. Tokens	4	4

Witten-Bell Smoothing (Unigram)

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1
No. Tokens	5	5
Uniq. Tokens	4	4
Lambda		

$$\lambda_c = \frac{T_c}{T_c + U_c}$$

Witten-Bell Smoothing (Unigram)

~~$$P(w|c) = \frac{\text{count}(w, c)}{T_c}$$~~

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1
No. Tokens	5	5
Uniq. Tokens	4	4
Lambda	0.56	0.56

$$\lambda_c = \frac{T_c}{T_c + U_c}$$

Witten-Bell Smoothing (Unigram)

~~$$P(w|c) = \frac{\text{count}(w, c)}{T_c}$$~~

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1
No. Tokens	5	5
Uniq. Tokens	4	4
Lambda	0.56	0.56
P(unseen c)		

$$unseen : \frac{1 - \lambda_c}{Z_c}$$

Witten-Bell Smoothing (Unigram)

~~$$P(w|c) = \frac{count(w, c)}{T_c}$$~~

$$P(w|c) = \begin{cases} seen : \lambda_c \cdot \frac{count(w, c)}{T_c} \\ unseen : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$count(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS
buy	1	0
cheap	2	1
medicines	1	0
bitcoin	1	0
client	0	1
meeting	0	2
morning	0	1
No. Tokens	5	5
Uniq. Tokens	4	4
Lambda	0.56	0.56
P(unseen c)	0.15	0.15

$$unseen : \frac{1 - \lambda_c}{Z_c}$$

$$unseen : \frac{1 - 0.56}{3}$$

Witten-Bell Smoothing (Unigram)

~~$$P(w|c) = \frac{count(w, c)}{T_c}$$~~

$$P(w|c) = \begin{cases} seen : \lambda_c \cdot \frac{count(w, c)}{T_c} \\ unseen : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$count(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS	P(w S)	P(w NS)
buy	1	0		
cheap	2	1		
medicines	1	0		
bitcoin	1	0		
client	0	1		
meeting	0	2		
morning	0	1		
No. Tokens	5	5		
Uniq. Tokens	4	4		
Lambda	0.56	0.56		
P(unseen c)	0.15	0.15		

Witten-Bell Smoothing (Unigram)

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS	P(w S)	P(w NS)
buy	1	0	0.56*0.2	
cheap	2	1	0.56*0.4	0.56*0.2
medicines	1	0	0.56*0.2	
bitcoin	1	0	0.56*0.2	
client	0	1		0.56*0.2
meeting	0	2		0.56*0.4
morning	0	1		0.56*0.2
No. Tokens	5	5		
Uniq. Tokens	4	4		
Lambda	0.56	0.56		
P(unseen c)	0.15	0.15		

Witten-Bell Smoothing (Unigram)

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS	P(w S)	P(w NS)
buy	1	0	0.112	
cheap	2	1	0.224	0.112
medicines	1	0	0.112	
bitcoin	1	0	0.112	
client	0	1		0.112
meeting	0	2		0.224
morning	0	1		0.112
No. Tokens	5	5		
Uniq. Tokens	4	4		
Lambda	0.56	0.56		
P(unseen c)	0.15	0.15		

Witten-Bell Smoothing (Unigram)

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS	P(w S)	P(w NS)
buy	1	0	0.112	
cheap	2	1	0.224	0.112
medicines	1	0	0.112	
bitcoin	1	0	0.112	
client	0	1		0.112
meeting	0	2		0.224
morning	0	1		0.112
No. Tokens	5	5		
Uniq. Tokens	4	4		
Lambda	0.56	0.56		
P(unseen c)	0.15	0.15		

Witten-Bell Smoothing (Unigram)

$$\cancel{P(w|c) = \frac{\text{count}(w, c)}{T_c}}$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS	P(w S)	P(w NS)
buy	1	0	0.112	0.150
cheap	2	1	0.224	0.112
medicines	1	0	0.112	0.150
bitcoin	1	0	0.112	0.150
client	0	1	0.150	0.112
meeting	0	2	0.150	0.224
morning	0	1	0.150	0.112
No. Tokens	5	5		
Uniq. Tokens	4	4		
Lambda	0.56	0.56		
P(unseen c)	0.15	0.15		

Witten-Bell Smoothing (Unigram)

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS	P(w S)	P(w NS)
buy	1	0	0.112	0.150
cheap	2	1	0.224	0.112
medicines	1	0	0.112	0.150
bitcoin	1	0	0.112	0.150
client	0	1	0.150	0.112
meeting	0	2	0.150	0.224
morning	0	1	0.150	0.112

$$P(\text{NotSpam} | \text{'bitcoin meeting'}) = 0.5 \cdot 0.150 \cdot 0.224$$

$$P(\text{Spam} | \text{'bitcoin meeting'}) = 0.5 \cdot 0.112 \cdot 0.150$$

Witten-Bell Smoothing (Unigram)

$$~~P(w|c) = \frac{\text{count}(w, c)}{T_c}~~$$

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Probability Smoothing

Token	S	NS	P(w S)	P(w NS)
buy	1	0	0.112	0.150
cheap	2	1	0.224	0.112
medicines	1	0	0.112	0.150
bitcoin	1	0	0.112	0.150
client	0	1	0.150	0.112
meeting	0	2	0.150	0.224
morning	0	1	0.150	0.112

$$P(\text{NotSpam} | \text{'bitcoin meeting'}) = 0.0168$$

$$P(\text{Spam} | \text{'bitcoin meeting'}) = 0.0084$$

Witten-Bell Smoothing (Unigram)

~~$$P(w|c) = \frac{\text{count}(w, c)}{T_c}$$~~

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Sparsity: What can we do about it?

Question

Use Witten-Bell smoothing to classify:
[hello, prince, client, office].

Probability Smoothing

Token	S	NS
hello	15	15
prince	12	1
princess	8	0
client	12	14
french	0	10
london	3	2
office	0	8

$$\lambda_c = \frac{T_c}{T_c + U_c}$$

Witten-Bell Smoothing (Unigram)

~~$$P(w|c) = \frac{\text{count}(w, c)}{T_c}$$~~

$$P(w|c) = \begin{cases} \text{seen} : \lambda_c \cdot \frac{\text{count}(w, c)}{T_c} \\ \text{unseen} : \frac{1 - \lambda_c}{Z_c} \end{cases}$$

λ_c : Witten-Bell weight.

$\text{count}(w, c)$: No. occurrences of w in class c .

T_c : No. tokens in class c .

U_c : No. unique tokens in class c .

Z_c : No. vocabulary words not present in class c .

Learning Objectives

Describe when and how sparsity can hinder representations.

Apply three methods to reduce sparsity.

Evaluate the impact of sparsity reduction methods on efficiency.

Feedback

- We have 10 weeks together, let me know how it is going!
- This is my first time running this course, so I am keen to hear your thoughts.
- It's never too early or too late to provide feedback.

<https://forms.gle/9WsY57cZKSkLQz8k6>



CM52065 Natural Language Processing

Hi folks,

Thanks for taking the time to complete this very short feedback form, this is all anonymous and will be checked regularly!

Thanks,
Andy

* Indicates required question

Feedback *

Your answer

Submit Clear form